

SIMPLEX Central Operations Console

OFFICIAL DOCUMENT

System Architecture, Deployment Guide & Operations Manual

Version 1.0.0 • Enterprise Edition

SIMPLEX Central Operations Console

Comprehensive Technical Documentation, Deployment Guide & Operations Manual

Executive Summary

The **SIMPLEX Central Operations Console** is an enterprise-grade, distributed operations and automation platform designed to centralize administration, identity lifecycle management, bulk data onboarding, resource tracking, and robotic process automation (RPA) across multiple remote **Simplex Hospital Information Management System (HIMS)** installations.

The platform bridges modern cloud/on-premise central management with legacy browser-driven clinical portals through high-resilience browser automation, cryptographic envelope credential protection, and transactional synchronization.

1. System Architecture & Monorepo Structure

The solution is structured as a TypeScript monorepo governed by `pnpm` workspaces:

```
SIMPLEX/
├── apps/
│   ├── api/           # Central REST API (NestJS, TypeORM, MSSQL, JWT, Swagger)
│   ├── web/           # Web Operations Portal (React 18, Vite, Tailwind CSS, Lucide)
│   └── desktop-agent/  # Automation Runner (CLI runner & Electron host with Playwright)
├── packages/
│   ├── automation/    # Browser RPA engine (Playwright, resilient selector engine)
│   ├── database/      # MSSQL database layer, TypeORM entities, envelope encryption
│   └── shared/         # Universal TypeScript contracts, DTOs, route resolvers
├── scripts/
│   └── start-all.js   # Unified multi-service orchestrator for development & staging
└── docker-compose.yml  # Containerized MSSQL Server database stack
```

Core Architecture Pillars:

- 1. **Central API Server (`apps/api`)**: Port `3000`. Exposes RESTful endpoints, manages JWT authentication, validates RBAC, enforces tenant isolation, handles background telemetry, and buffers multi-chunk sync batches.
- 2. **Web Operations Console (`apps/web`)**: Port `5173`. Responsive Single Page Application (SPA) offering immediate visibility into synchronized clients, live user directories, resource hierarchies, import pipelines, and agent heartbeats.
- 3. **Desktop Automation Agent (`apps/desktop-agent`)**: Connects to the Central API via secure pairing token, continuously polls for dispatched tasks (250ms event-poller), launches isolated headless or

headed Chromium browser contexts, interacts with remote Simplex DOM elements, and streams progress telemetry.

4. **Resilient Automation Engine** (`packages/automation`): Houses multi-tier selector engines with fallbacks tailored for Simplex versions (e.g., `MasterV9.1` , `MasterV9.4` , `EDSC`), handling dynamic Angular `$digest` cycles, iframe nesting, anti-tamper inputs, and modal popups.
5. **Database & Security Layer** (`packages/database`): Built on Microsoft SQL Server. Implements **AES-256-GCM Envelope Encryption** with PBKDF2 key derivation so remote hospital administrator passwords are never stored in plain text.

2. System Configuration & Resource Sizing Guide

2.1 Development vs. Production Footprint (Why Dev Takes More RAM)

On a local development workstation, memory usage often reaches **14–16 GB** due to developer tools:

- **Docker Desktop Virtual Machine**: Pre-allocates a fixed **8 GB RAM** virtual machine by default.
- **IDE & Language Indexers (VS Code / Antigravity)**: Consumes **2–3 GB RAM** for AST indexing, TypeScript typechecking, and syntax trees.
- **Development Compilers & Watchers**: Vite dev servers, Next.js servers, and Webpack HMR watchers consume **1.5–2 GB RAM**.

In a **Production Environment**, none of these development tools exist! The application runs **pre-compiled artifacts**:

- **Central API** (`node dist/main.js`): Only consumes **~120 MB – 180 MB RAM**.
- **Web Operations Portal** (`dist/` static files via Nginx/Caddy): Only consumes **~15 MB – 25 MB RAM** (static assets, zero Node overhead).
- **Desktop Automation Runner** (`node dist/cli-runner.js`): Consumes **~75 MB RAM idle**, and **~250 MB – 350 MB RAM** during an active headless Playwright run (freed immediately upon task completion).
- **Database (MSSQL Express Edition or Managed Cloud Database)**: Consumes **~1.0 GB – 1.5 GB RAM** (or **0 MB local RAM** if using Azure SQL / Amazon RDS).

2.2 Production Sizing Profiles

Profile A: Ultra-Lightweight Production (Single VM / Basic Server)

[!TIP] Recommended for budget deployments, small/medium hospital networks, or single-facility installations.

- **CPU: 2 vCPUs** (x86_64 or ARM64)
- **Physical RAM: 4 GB RAM** total
- **Storage: 25 GB SSD**
- **Deployment Model:**
 - Central API running via `PM2` (~150 MB)

- Web Console served via Nginx or NestJS static serving (~20 MB)
 - Desktop Agent CLI Runner (~100 MB idle, ~300 MB during runs)
 - Managed Cloud Database (Azure SQL / AWS RDS) **OR** local MSSQL Express Edition (memory capped at 1.4 GB)
- **Approximate Cloud Cost:** ~\$10 – \$20 / month on AWS, DigitalOcean, Hetzner, or Azure.

Profile B: Standard Enterprise Production (Multi-Hospital Fleet)

[!NOTE] Recommended for large hospital networks managing 10+ client instances with concurrent automation runners.

- **CPU: 4 vCPUs to 8 vCPUs**
- **Physical RAM: 8 GB to 16 GB RAM**
- **Storage: 50 GB to 100 GB NVMe SSD**
- **Deployment Model:**
 - Central API containerized with auto-scaling
 - Dedicated high-availability MSSQL 2022 instance
 - Multiple distributed Desktop Agent runner nodes processing hospital queues in parallel

3. Step-by-Step Setup & Deployment Guide (New System)

Step 1: Install Node.js and PNPM

```
# Verify or install Node.js (v20+ required)
node -v

# Install pnpm globally
npm install -g pnpm
pnpm -v
```

Step 2: Install Playwright Browser Binaries & Dependencies

```
# Install Playwright browser dependencies (Chromium)
npx playwright install --with-deps chromium
```

Step 3: Configure Database (Microsoft SQL Server)

You can run MSSQL either locally, on a remote server, or using Docker:

```
# Example running MSSQL Server in Docker:
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=YourStrongPassword123" \
  -p 1433:1433 --name mssql-hmc --restart unless-stopped \
  -d mcr.microsoft.com/mssql/server:2022-latest
```

Ensure a database named `HMC_CENTRAL_AUTOMATION` is created (or let the migration scripts auto-initialize).

Step 4: Configure Environment Variables

Create a `.env` file in the project root (`/SIMPLEX/.env`):

```
# Microsoft SQL Server Database Configuration
MSSQL_HOST=localhost
MSSQL_PORT=1433
MSSQL_DATABASE=HMC_CENTRAL_AUTOMATION
MSSQL_USER=sa
MSSQL_PASSWORD=YourStrongPassword123
MSSQL_ENCRYPT=false
MSSQL_TRUST_SERVER_CERTIFICATE=true

# API Server Configuration
PORT=3000
NODE_ENV=production
API_BASE_URL=http://localhost:3000
WEB_BASE_URL=http://localhost:5173

# Security & Cryptography
ENCRYPTION_MASTER_KEY=e8b839655f46a7be7e3c15c6b7582b1c853f6517a942bcba5e7e600d89e574ac
JWT_SECRET=hmc_central_jwt_secret_dev_key_2026_super_secure_token
JWT_EXPIRES_IN=24h
JWT_REFRESH_SECRET=hmc_central_refresh_secret_dev_key_2026_super_secure_token
JWT_REFRESH_EXPIRES_IN=7d

# Account Security & Lockout
MAX_FAILED_LOGIN_ATTEMPTS=5
LOCKOUT_DURATION_MINUTES=15

# Desktop Agent Configuration
AGENT_SHARED_SECRET=hmc_agent_shared_secret_pair_key_2026
AGENT_HEARTBEAT_INTERVAL_MS=10000

# Retention Policies (Days)
RETENTION_SCREENSHOTS_DAYS=30
RETENTION_ERROR_LOGS_DAYS=90
RETENTION_AUDIT_LOGS_DAYS=365
RETENTION_IMPORT_SUMMARIES_DAYS=365
```

Step 5: Install Dependencies and Build All Packages

```
# From the repository root
pnpm install

# Build all libraries and applications
pnpm build
```

Step 6: Initialize Database Migrations & Default Admin

```
# Run schema migrations and base seeds
pnpm db:migrate
pnpm db:seed
pnpm admin:bootstrap
```

Default Seeded Super Admin Credentials:

- **Username:** `admin`

- **Password:** `Rajesh@123` (Prompted to rotate or changeable in settings)

Step 7: Launch All Services

To start all three tiers (API, Web UI, Desktop Agent) in a single unified process:

```
pnpm start
# OR: node scripts/start-all.js
```

Alternatively, to run each service independently (e.g. across separate PM2 / systemd daemons):

```
# Terminal 1 – Central API:
pnpm --filter @hmc/api start

# Terminal 2 – Web Console:
pnpm --filter @hmc/web dev -- --host 0.0.0.0 --port 5173

# Terminal 3 – Desktop Automation Agent:
pnpm --filter @hmc/desktop-agent start:runner
```

4. User Concurrency & System Capacity

4.1 Web Console & Central API Concurrency

- **Concurrent Interactive Operators: 500 to 2,000+ simultaneous users.**
- **Architecture:** The Central API is built on NestJS running on Node.js non-blocking asynchronous event loop. Session state is entirely **stateless JWT**, requiring zero server-side session memory per connected web operator.
- **Database Connection Pooling:** MSSQL connection pooling is managed by `Tedious` and `TypeORM` with configurable minimum and maximum pool bounds (default: 10 min, 50 max). Read queries for cached directory snapshots resolve in **< 15 milliseconds**.
- **Horizontal Scalability:** The Central API can be containerized and load-balanced behind Nginx, AWS ALB, or Cloudflare with zero session sticky requirements.

4.2 Automation Agent Concurrency & Queue Model

- **Task Queuing:** Central API maintains an in-memory and database-backed FIFO task queue per registered client.
- **Single-Flight Lock (Per Client):** To prevent session collision and race conditions on remote Simplex hospital portals, the platform enforces a **single-flight lock per client instance profile**. One automation task executes on a target Simplex portal at any given instant.
- **Batch Scrape Throughput:**
 - Scrapes **180 – 250 users in 6 to 12 seconds**.
 - Chunks users into bounded UTF-8 batches (100 users / batch, < 400 KB) for transactional insertion.
- **Multi-Agent Scale-Out:**
 - You can deploy multiple Desktop Agents across different machines or virtual machines.
 - Each agent pairs with the Central API and claims available tasks across separate client instances simultaneously. 10 paired agents can process 10 distinct hospital clients in parallel.

5. Comprehensive Screen-by-Screen Functionality Guide

SIMPLEX Central Operations Console: Screen Directory		
1. /login	- Secure Authentication & Session Gateway	
2. / (Dashboard)	- Central Operations Cockpit & Health Telemetry	
3. /clients	- Remote Hospital Client Instances & Key Vault	
4. /users	- Client Users Directory & Remote Operations	
5. /imports	- Bulk User & Resource Data Ingestion Pipelines	
6. /resources	- Clinical Resource Directory, Specialties & Departments	
7. /roles	- Role-Based Access Control (RBAC) & Permissions Matrix	
8. /agents	- Desktop Automation Agent Fleet & Task Telemetry	
9. /audit	- Comprehensive Compliance Audit Trails & Export	
10. /settings	- System Parameters, Retention Rules & Password Policies	

Screen 1: Login & Session Gateway (/login)

- **Primary Purpose:** Authenticate operational personnel with multi-tier credentials, enforce security lockouts, and initiate JWT bearer token sessions.
- **Key Capabilities:**
 - **Brute-Force Protection:** Tracks failed attempts per username and client IP. Locks account for 15 minutes after 5 consecutive failed attempts.
 - **Password Rotation Notice:** Detects default initial passwords and guides operator to mandatory password update.
 - **Session Longevity:** Access token valid for 24 hours; refresh token rotation valid for 7 days.
 - **Role-Based Redirect:** Super Admins, Client Managers, and Auditors are automatically routed to their authorized views.

Screen 2: Central Operations Dashboard (/)

- **Primary Purpose:** Real-time high-level operations cockpit providing immediate situational awareness across all connected Simplex hospitals.
- **Key Capabilities:**
 - **Summary KPI Cards:** Displays count of Active Client Hospitals, Total Synchronized Users, Active Desktop Agents, and Recent Automation Success Rates.
 - **Active Execution Feed:** Live visual feed of in-flight sync jobs, password resets, and user creations.
 - **Quick Navigation Shortcuts:** One-click jump to sync users, launch an interactive browser session, or review audit logs.
 - **System Health Badge:** Indicates Central API connectivity, database responsiveness, and paired agent availability.

Screen 3: Remote Hospital Client Instances (/clients)

- **Primary Purpose:** Register, configure, and maintain individual Simplex hospital installations.

- **Key Capabilities:**

- **Client Registration Modal:** Capture Client Name, Unique Client Code (e.g. `MASTER9.1`, `EDSC`), and Base URL (`https://staging.simplexworld.com/MasterV9.1`).
- **Application Path & Version Customization:** Specify custom root paths (`applicationPath`), login routes (`/login`), and user directory routes (`/users` or `/addUsers`).
- **Encrypted Credential Vault:**
 - Store administrative credentials (`ADMIN` username and password) used by the automation agent to log in.
 - Credentials are encrypted using **AES-256-GCM** with individual IV and authentication tags.
- **Test Connection:** Dispatches a lightweight headless check to confirm whether the URL is reachable and the credentials authenticate successfully.
- **Open Interactive Browser Session:** Dispatches a command to the Desktop Agent to open a visible, dedicated Chrome window pre-logged into that client's Simplex portal.

Screen 4: Client Users Directory & Remote Operations (`/users`)

- **Primary Purpose:** The core operational interface for managing hospital personnel accounts across any selected Simplex client.
- **Key Capabilities:**
 - **Target Client Switcher:** Filter the entire directory by selecting any registered Simplex client.
 - **Live Remote Synchronization (`Sync Users` Button):**
 - Triggers a headless background automation job (`SYNC_CLIENT_USERS_HEADLESS`).
 - The Desktop Agent logs into the remote Simplex portal, extracts the complete user table across pagination, validates records, and transmits byte-aware batches to Central API.
 - Updates directory snapshots with zero downtime.
 - **Remote Status Toggle (ACTIVE / INACTIVE):**
 - Toggle user status directly from the table.
 - Agent launches visible Chrome context, searches user on Simplex, modifies dropdown/checkbox status, submits change, verifies DOM updated, and reconciles database.
 - **Remote Password Reset:**
 - Click `Reset Password` on any user row.
 - Agent opens Simplex password reset dialog, enters a cryptographically secure temporary password, and verifies successful submission.
 - Displays the **One-Time Ephemeral Credential Delivery Modal:** Allows the operator to copy the temporary password once. Never stored in plain text.
 - **Search & Filters:** Instant search by Username, Full Name, Mobile Number, or Status.
 - **Role Mapping:** Inspect and update assigned user roles directly from the UI.
 - **Excel Export:** Export the full synchronized user directory to `.xlsx` .

Screen 5: Bulk Data Import & Ingestion Pipelines (`/imports`)

- **Primary Purpose:** High-volume batch onboarding of hospital users and clinical resources via Excel spreadsheets.
 - **Key Capabilities:**
 - **Excel Template Download:** Generates pre-formatted `.xlsx` templates with required columns, validation rules, and sample data.
 - **File Upload & Pre-Validation Staging:**
 - Validates duplicate usernames, missing mobile numbers, invalid characters, and role existence *before* touching remote Simplex servers.
 - Categorizes rows into **Valid**, **Warning**, and **Error**.
 - **Execution Controller:** Start, Pause, Resume, or Cancel import jobs.
 - **Error Export:** Download filtered error spreadsheets containing exact row-by-row failure reasons for quick correction.
-

Screen 6: Clinical Resources Directory (`/resources`)

- **Primary Purpose:** Manage hospital doctors, clinicians, consulting rooms, and medical equipment resources.
 - **Key Capabilities:**
 - **Human vs Non-Human Classification:** Toggle between human practitioners (Doctors, Specialists) and non-human assets (Surgical Theaters, Beds, Equipment).
 - **Specialty & Department Hierarchies:** View and manage assigned clinical departments, services, and EMR forms.
 - **Map User to Resource:** Link a registered application user account to their clinical resource profile.
 - **Sync Resources:** Dispatches automated scraper targeting Simplex `ResourceParent` route to discover all registered resources.
-

Screen 7: Desktop Automation Agents Fleet (`/agents`)

- **Primary Purpose:** Monitor health, telemetry, and status of connected automation agents.
 - **Key Capabilities:**
 - **Agent Fleet Table:** Displays Agent Name, Machine Hostname, Operating System, Status (`ONLINE` , `BUSY` , `OFFLINE`), and Last Heartbeat Timestamp.
 - **Active Task Inspection:** Shows which task is currently running on each agent, elapsed execution time, and real-time step progress.
 - **Agent Pairing Token Generator:** Generate pairing secrets to register new desktop agents onto the central cluster.
-

Screen 8: Role-Based Access Control (RBAC) (`/roles`)

- **Primary Purpose:** Define granular security permissions for central console operators.
- **Key Capabilities:**
 - **Predefined System Roles:** `SUPER_ADMIN` , `CLIENT_OPERATOR` , `AUDITOR` , `READ_ONLY` .

- **Granular Permissions Matrix:** Toggle rights for:
 - `CLIENTS_READ` , `CLIENTS_WRITE` , `CLIENTS_CREDENTIALS`
 - `CLIENT_USERS_READ` , `CLIENT_USERS_SYNC` , `CLIENT_USERS_MUTATE`
 - `IMPORTS_CREATE` , `IMPORTS_EXECUTE`
 - `AUDIT_LOGS_VIEW` , `AUDIT_LOGS_EXPORT`
 - `AGENTS_MANAGE`
 - **Client Access Scope:** Limit operators so they can only view and manage specific hospital clients.
-

Screen 9: Audit Logs & Security Trails (`/audit`)

- **Primary Purpose:** Complete regulatory compliance and forensic audit logging.
 - **Key Capabilities:**
 - **Tamper-Evident Event Log:** Records every single login, client modification, credential access, sync execution, user status change, and password reset.
 - **Event Metadata:** Captures Timestamp, Actor ID, Actor Username, Action Code, Target Entity, Outcome (`SUCCESS` / `FAILURE`), IP Address, and Correlation UUID.
 - **Regulatory Export:** Export filtered audit trails to CSV/Excel for compliance reporting (HIPAA / ISO 27001 / SOC 2).
-

Screen 10: Settings & Retention Policies (`/settings`)

- **Primary Purpose:** System configuration, retention lifecycle management, and security policies.
 - **Key Capabilities:**
 - **Data Retention Rules:** Configure automated purge cycles (Days) for error logs, automation run screenshots, completed import job rows, and historical telemetry.
 - **Manual Purge Trigger:** Clean up historical ephemeral data on demand.
 - **Session Security Controls:** Adjust token timeouts, max failed login attempts, and password complexity enforcement.
-

6. Verification & Healthcheck Endpoints

The Central API provides built-in health and readiness endpoints for automated monitoring and container orchestrators:

- **Live Check:** `GET http://localhost:3000/api/v1/health/live`
 - Returns `200 OK` if the process is responsive.
- **Ready Check:** `GET http://localhost:3000/api/v1/health/ready`
 - Validates active MSSQL database connection pool and TypeORM responsiveness.
- **Swagger API Documentation:** `GET http://localhost:3000/api/docs`
 - Complete interactive OpenAPI specification for all Central endpoints.